CUET PG

DAILY CLASS NOTES

Computer Science

Operating System

Lecture – 13

Paging and Segmentation: Part 2



Paging and Segmentation: Part 2

Topics to be Covered

1. Multiprogramming with Variable Partition
2. Paging
3. Segmentation

1. Multiprogramming with Variable Partition

Static Partitioning

- ★ Static partitioning works well in **fixed and predictable workloads**.
- ★ It is **unsuitable** when program memory requirements vary or are unpredictable.
- ★ Major disadvantages:
  - ○ **Inflexibility** – cannot adjust to dynamic workloads.
  - ○ **Internal Fragmentation** – unused memory space inside a partition.

Dynamic (Variable) Partitioning

- ★ To overcome static partitioning issues, **variable partitioning** is used.
- ★ **Partitions are created dynamically** instead of being fixed beforehand.
- ★ Each process is allocated **exactly the amount of memory it needs** — no more, no less.
- ★ However, over time, **external fragmentation** occurs because free memory (holes) becomes scattered.

External Fragmentation

- ★ It means free memory blocks between allocated areas become **non-contiguous**, making it difficult to allocate large processes.

Compaction

- ★ **Compaction** is a technique to overcome external fragmentation.
- ★ It involves **relocating processes** in memory so that free memory blocks (holes) are combined into a single large block.
- ★ This allows other processes to get contiguous free memory.

Algorithms for Allocating Free Memory (Hole Selection)

Depending on the system's strategy, **three algorithms** can be used:

Algorithm	Description	Remarks
First Fit	Allocate the **first hole** that is big enough for the process.	Fast but can cause scattered small holes.
Next Fit	Variant of First Fit – searching starts **from where the previous search ended**.	Slightly improves distribution.
Best Fit	Allocate the **smallest available hole** that is still large enough.	Produces **least leftover space** (less fragmentation).
Worst Fit	Allocate the **largest hole** available.	Produces the **largest leftover gaps** (least efficient).

Detailed Algorithm Explanation

(a) First Fit:

- ★ Scans the memory list from the beginning.
- ★ Allocates the first suitable free hole.
- ★ Stops when a sufficient space is found.

(b) Best Fit:

- ★ Scans the entire list to find the **smallest sufficient hole**.
- ★ Produces **small unused spaces**, reducing fragmentation.

(c) Worst Fit:

- ★ Choose the **largest hole**.
- ★ Creates the largest remaining unused space (least efficient).

Comparison Note

- ★ **First Fit** and **Best Fit** are better than **Worst Fit** in terms of **memory and time utilisation**.
- ★ However, both still cause **external fragmentation**.
- ★ **Compaction** can reduce fragmentation but is **time-consuming**.
- ★ Another effective solution is to **allow non-contiguous memory allocation**, which is achieved through **paging**.

2. Paging

Definition

- ★ **Paging** is a **memory management scheme** that removes the need for contiguous memory allocation.
- ★ It allows a process's **physical memory** to be **non-contiguous**

Concept

- ★ **Physical memory** is divided into **fixed-size blocks** called **frames**.
- ★ **Logical memory (virtual address space)** is divided into the **same-size blocks** called **pages**.
- ★ Each page of a process is stored in any available frame in memory.

Page Map Table (PMT)

- ★ Paging uses a **mapping table** called the **Page Map Table (PMT)** for address translation.
- ★ The **PMT** is created when a process is loaded.
- ★ It maintains a **mapping between logical (virtual) pages and physical frames**.

Component	Description
Page Number	Index in the page table (logical page ID).
Frame Number	Actual frame in physical memory.
PMT Entry	Stores that frame correspond to which page.

Each page entry in PMT points to the corresponding frame number in physical memory.

Logical Address Structure

Page Number (P)	Page Offset (D)
M - n bits	N bits

- ★ **P**: Index in page table.
- ★ **D**: Offset within the page (displacement).
- ★ Total logical address = **Page number + offset**.

Note:

- ★ If the page size is **not a power of 2**, separation of P and D is **not possible**.

3. Segmentation

Definition

- ★ **Segmentation** is another **non-contiguous memory allocation technique**.
- ★ Difference from Paging:
  - ○ **Paging** divides memory into **fixed-size physical blocks**.
  - ○ **Segmentation** divides a program into **logical blocks (segments)** of **variable size**.

Concept

- ★ Logical address space is divided into segments such as:
  - ○ **Main program**
  - ○ **Subroutines or functions**
  - ○ **Procedures**
  - ○ **Data structures (stack, array, etc.)**
- ★ Each **segment has a name and length**.
- ★ Loaded into physical memory **as it is** (not divided).
- ★ For simplicity, segments are represented by **segment numbers** rather than names.

Logical Address Format

A logical address in segmentation is represented as:

→ **<segment_number, offset>**

Field	**Meaning**
Segment Number	Identifies the segment.
Offset	Specifies location within the segment.

4. Segmentation with Paging (Combined Approach)

Concept

- ★ Both **paging** and **segmentation** have advantages and limitations.
- ★ Neither is completely superior, so **a combination** of both is often used.

Working of the Combined Scheme

- ★ From the **user's point of view**, memory is divided into **logical segments**.
- ★ From the **system's point of view**, each segment is further divided into **pages** of fixed size.

Advantages

- ★ Combines **logical convenience of segmentation** with the **physical efficiency of paging**.
- ★ Avoids **external fragmentation** (paging advantage).
- ★ Reduces **complex segment management** and **secondary memory allocation issues**.

पेजिंग और सेगमेंटेशन: भाग 2

कवर किए जाने वाले विषय

1. वेरिएबल पार्टीशन के साथ मल्टीप्रोग्रामिंग
2. पेजिंग
3. सेगमेंटेशन

1. वेरिएबल पार्टीशन के साथ मल्टीप्रोग्रामिंग

स्टेटिक पार्टिशनिंग

★ स्थिर विभाजन स्थिर और पूर्वानुमानित कार्यभार में अच्छा काम करता है।
★ यह तब अनुपयुक्त होता है जब प्रोग्राम की मेमोरी आवश्यकताएं बदलती रहती हैं या अप्रत्याशित होती हैं।
★ प्रमुख कमियां:
  ○ **फ्लेक्सिबल** – गतिशील कार्यभार के अनुसार समायोजित नहीं हो सकती।
  ○ **इंटर्नल फ्रेगमेंटेशन** – विभाजन के भीतर अप्रयुक्त मेमोरी स्थान।

डायनामिक (वेरीयबल) पार्टिशनिंग

★ स्टैटिक पार्टिशनिंग संबंधी समस्याओं को दूर करने के लिए, वेरिएबल पार्टिशनिंग का उपयोग किया जाता है।
★ इसमें विभाजन पहले से तय नहीं होते, बल्कि गतिशील रूप से बनाए जाते हैं।
★ प्रत्येक प्रक्रिया को ठीक उतनी ही मेमोरी एलोकेशन की जाती है जितनी उसे आवश्यकता होती है–न अधिक, न कम।
★ हालांकि, समय के साथ, एक्सटर्नल फ्रेगमेंटेशन होता है क्योंकि खाली मेमोरी (रिक्त स्थान) बिखर जाती है।

एक्सटर्नल फ्रेगमेंटेशन

★ इसका मतलब है कि एलोकेशन क्षेत्रों के बीच मुक्त मेमोरी ब्लॉक असंगत हो जाते हैं, जिससे बड़ी प्रक्रियाओं को एलोकेशन करना मुश्किल हो जाता है।

कॉम्पैक्शन

★ कंपैक्शन बाह्य विखंडन को दूर करने की एक तकनीक है।
★ इसमें मेमोरी में प्रक्रियाओं को इस प्रकार स्थानांतरित किया जाता है कि खाली मेमोरी ब्लॉक (होल्स) एक बड़े ब्लॉक में संयोजित हो जाते हैं।
★ इससे अन्य प्रक्रियाओं को निरंतर खाली मेमोरी प्राप्त हो जाती है।

मुक्त मेमोरी एलोकेशन करने के लिए एल्गोरिदम (होल चयन)

सिस्टम की रणनीति के आधार पर, तीन एल्गोरिथम का उपयोग किया जा सकता है:

एल्गोरिथम	विवरण	टिप्पणी
फर्स्ट फिट	प्रक्रिया के लिए पर्याप्त बड़ा पहला छिद्र एलोकेट करें।	तेज़ गति से काम करता है लेकिन इससे जगह-जगह छोटे-छोटे छिद्र हो सकते हैं।
नेक्स्ट फिट	फर्स्ट फिट का एक प्रकार – खोज वहीं से शुरू होती है जहाँ पिछली खोज समाप्त हुई थी।	वितरण में थोड़ा सुधार होता है।
बेस्ट फिट	उपलब्ध सबसे छोटे छिद्र का एलोकेट करें जो अभी भी पर्याप्त बड़ा हो।	इससे सबसे कम शेष स्थान उत्पन्न होता है (कम विखंडन)।
वर्स्ट फिट	उपलब्ध सबसे बड़े छिद्र को एलोकेशन करें।	इससे सबसे अधिक शेष अंतराल उत्पन्न होते हैं (सबसे कम कुशल)।

डिटेल्ड एल्गोरिथम व्याख्या

(a) फर्स्ट फिट:

- ★ मेमोरी सूची को शुरुआत से स्कैन करता है।
- ★ सबसे उपयुक्त खाली स्थान एलोकेट करता है।
- ★ पर्याप्त स्थान मिलने पर रुक जाता है।

(b) बेस्ट फिट:

- ★ सबसे छोटे उपयुक्त रिक्त स्थान को खोजने के लिए पूरी सूची को स्कैन करता है।
- ★ छोटे अप्रयुक्त स्थान उत्पन्न करता है, जिससे फ्रेगमेंटेशन कम होता है।

(c) वर्स्ट फिट:

- ★ सबसे बड़ा छिद्र चुनें।
- ★ इससे सबसे अधिक अप्रयुक्त स्थान बचता है (सबसे कम कुशल)।

तुलना नोट

- ★ मेमोरी और समय के उपयोग के मामले में फर्स्ट फिट और बेस्ट फिट, वर्स्ट फिट से बेहतर हैं।
- ★ हालांकि, दोनों ही एकस्टर्नल फ्रेगमेंटेशन का कारण बनते हैं।
- ★ संपीड़न से विखंडन कम हो सकता है, लेकिन इसमें समय लगता है।
- ★ एक अन्य प्रभावी समाधान है नॉन-कंटीग्यूअस मेमोरी आवंटन की अनुमति देना, जो पेजिंग के माध्यम से प्राप्त किया जाता है।

2. पेजिंग

परिभाषा

- ★ पेजिंग एक मेमोरी मैनेजमेंट प्रणाली है जो निरंतर मेमोरी एलोकेशन की आवश्यकता को समाप्त करती है।
- ★ यह किसी प्रक्रिया की भौतिक मेमोरी को नॉन-कंटीग्यूअस होने की अनुमति देती है।

अवधारणा

- ★ फिजिकल मेमोरी को निश्चित आकार के ब्लॉकों में विभाजित किया जाता है जिन्हें फ्रेम कहा जाता है।
- ★ लॉजीकल मेमोरी (वर्चुअल एड्रेस स्पेस) को समान आकार के ब्लॉकों में विभाजित किया जाता है जिन्हें पेज कहते हैं।
- ★ किसी प्रक्रिया का प्रत्येक पेज मेमोरी में उपलब्ध किसी भी फ्रेम में संग्रहीत होता है।

पेज मैप टेबल (PMT)

- ★ पेजिंग एड्रेस के अनुवाद के लिए पेज मैप टेबल (PMT) नामक मैपिंग टेबल का उपयोग करती है।
- ★ PMT किसी प्रक्रिया के लोड होने पर बनाई जाती है।
- ★ यह लॉजिकल (वर्चुअल) पेजों और फिजिकल फ्रेम के बीच मैपिंग बनाए रखती है।

घटक	विवरण
पेज नंबर	पेज टेबल में इंडेक्स (लॉजिकल पेज आईडी)।
फ्रेम नंबर	फिजिकल मेमोरी में वास्तविक फ्रेम।
PMT एंट्री	वे स्टोर जो फ्रेम को दर्शाते हैं, वे किस पेज से मेल खाते हैं।

PMT में प्रत्येक पेज प्रविष्टि भौतिक मेमोरी में संबंधित फ्रेम संख्या को इंगित करती है।

★ **लॉजिकल एड्रेस स्ट्रक्चर**

पेज नंबर (P)	**पेज Offset (D)**
M - n बिट	N बिट

★ P: पेज टेबल में इंडेक्स।

★ D: पेज के भीतर ऑफ़सेट (डिस्प्लेसमेंट)।

★ कुल लॉजिकल एड्रेस = पेज नंबर + ऑफ़सेट।

नोट:

★ यदि पेज का आकार 2 की घात नहीं है, तो P और D को अलग करना संभव नहीं है।

3. सेगमेंटेशन

परिभाषा

★ सेगमेंटेशन एक अन्य नॉन-कंटीगुअस मेमोरी एलोकेशन तकनीक है।

★ **पेजिंग से अंतर:**

- पेजिंग मेमोरी को निश्चित आकार के भौतिक ब्लॉकों में विभाजित करती है।
- सेगमेंटेशन एक प्रोग्राम को परिवर्तनीय आकार के तार्किक ब्लॉकों (सेगमेंट) में विभाजित करती है।

अवधारणा

★ लॉजिकल एड्रेस स्थान को निम्नलिखित खंडों में विभाजित किया गया है:

- मेन प्रोग्राम
- सबरूटीन या फ़ंक्शन
- प्रक्रियाएँ
- डेटा स्ट्रक्चर (स्टैक, ऐरे, आदि)

★ प्रत्येक सेगमेंट का एक नाम और लंबाई होती है।

★ इसे फिजिकल मेमोरी में यथावत (विभाजित किए बिना) लोड किया जाता है।

★ सरलता के लिए, सेगमेंट को नामों के बजाय सेगमेंट संख्याओं द्वारा दर्शाया जाता है।

लॉजिकल एड्रेस फॉर्मेट

सेगमेंटेशन में लॉजिकल एड्रेस को इस प्रकार दर्शाया जाता है:

→ <सेगमेंट_नंबर, ऑफ़सेट>

क्षेत्र	**अर्थ**
सेगमेंट संख्या	यह सेगमेंट की पहचान करता है।
ऑफसेट (Offset)	सेगमेंट के भीतर स्थान निर्दिष्ट करता है।

4. पेजिंग के साथ सेगमेंटेशन (संयुक्त दृष्टिकोण)

अवधारणा

★ पेजिंग और सेगमेंटेशन दोनों के अपने-अपने फायदे और नुकसान हैं।

★ दोनों में से कोई भी पूरी तरह से श्रेष्ठ नहीं है, इसलिए अक्सर दोनों का संयोजन उपयोग किया जाता है।

संयुक्त योजना की कार्यप्रणाली

★ उपयोगकर्ता के दृष्टिकोण से, मेमोरी को लॉजिकल सेगमेंट में विभाजित किया जाता है।

★ सिस्टम के दृष्टिकोण से, प्रत्येक खंड को निश्चित आकार के पृष्ठों में विभाजित किया जाता है।

लाभ

★ यह विभाजन की तार्किक सुविधा को पेजिंग की भौतिक दक्षता के साथ जोड़ता है।

★ यह एकस्टर्नल फ्रेगमेंटेशन से बचाता है (पेजिंग का लाभ)।

★ यह कॉम्प्लेक्स सेगमेंट मैनेजमेंट और सेकेंडरी मेमोरी एलोकेशन संबंधी समस्याओं को कम करता है।



PW Web/App: https://smart.link/7wwosivoicgd4

Library- https://smart.link/sdfez8ejd80if